

Machine Learning Fundamentals

Hari 1 · Navasena Gen-ML Course · Batch 6

10 notebook · Python · Google Colab

Navasena AI

Peta hari ini

Regresi

nb01 Linear Regression

Klasifikasi

nb02 Logistic Regression
nb03 Decision Tree
nb04 KNN
nb05 SVM

Ensemble

nb06 Voting, bagging,
boosting
nb07 XGBoost

Clustering

nb08 K-Means, hierarchical,
DBSCAN

Time series

nb09 Dekomposisi dan
SARIMA

GPU

nb10 cuML dan XGBoost
di GPU

Sebelum notebook pertama kita bahas alur kerjanya dulu: bagaimana model belajar, dan bagaimana hasilnya diuji.

Act 1

Apa itu Machine Learning?

Kalau aturannya tidak bisa ditulis, biarkan datanya yang bicara

Spam filter versi if/else

- Subjek memuat “GRATIS” → dikirim sebagai “G R A T I S”
- Pengirim tidak dikenal → alamat baru dibuat tiap hari
- Lima tanda seru → dikurangi jadi dua

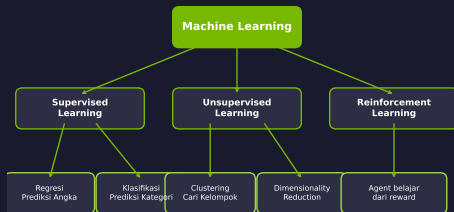
Versi machine learning

- 10.000 email yang sudah ditandai spam atau bukan
- Model menyesuaikan parameternya untuk menangkap pola pada contoh itu
- Contoh baru masuk, polanya ikut diperbarui

Intinya

Machine learning adalah program yang aturannya dipelajari dari contoh, bukan ditulis tangan satu per satu.

Tiga jenis machine learning



- **Supervised**: setiap contoh punya jawabannya. Sembilan notebook pertama ada di sini.
- **Unsupervised**: tanpa jawaban, model mencari struktur dalam data. Ini clustering di nbo8.
- **Reinforcement**: belajar dari reward atas tindakannya. Hari ini hanya sebagai konteks.

Contoh yang kamu pakai tiap hari

Layanan	Yang diprediksi	Jenis tugas
Gmail	email ini spam atau bukan	klasifikasi
Google Maps	lama waktu tempuh	regresi
Spotify	pendengar dengan selera mirip	clustering
Toko online	permintaan minggu depan	time series

Keempat baris ini memakai kelompok model yang kita buka hari ini juga. Bedanya ada di ukuran data dan jumlah orang yang merawat sistemnya.

Tools hari ini

- **Python** — bahasa yang dipakai sepanjang hari
- **Google Colab (GPU T4)** — menjalankan notebook tanpa instalasi di laptop
- **pandas, matplotlib** — memuat data, melihat isinya, membuat grafik
- **scikit-learn** — hampir semua model klasik di modul ini
- **XGBoost** — boosting untuk data tabular (nb07)
- **cuML** — model klasik yang dijalankan di GPU (nb10)

Semua notebook memuat datanya langsung dari URL, jadi tidak ada file yang perlu kamu unggah dulu.

Act 2

Bagaimana model belajar dan diuji?

Dari kolom data mentah sampai satu angka yang layak dilaporkan

Feature dan target

TV	radio	newspaper	sales
230,1	37,8	69,2	22,1
44,5	39,3	45,1	10,4
17,2	45,9	69,3	9,3

tiga feature target

- **Feature** adalah kolom masukan yang dipakai model: belanja iklan di tiga kanal.
- **Target** adalah nilai yang mau diprediksi: penjualan.
- Targetnya berupa angka, jadi tugasnya regresi. Kalau targetnya kategori, itu klasifikasi dan targetnya disebut label kelas.

Soal latihan

- Dikerjakan berulang sampai polanya kelihatan
- Jawabannya sudah ada di buku

Soal ujian

- Belum pernah dilihat sebelumnya
- Nilainya baru berarti kalau soalnya memang baru

Intinya

Data train dipakai untuk melatih model. Data test disimpan utuh dan dipakai sekali di akhir untuk menilai hasilnya.

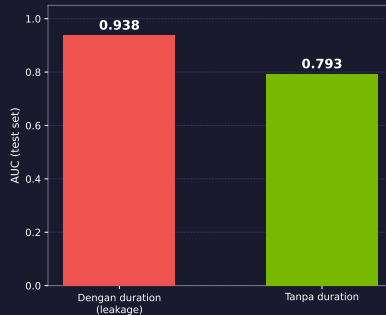
Data train, validation, dan test



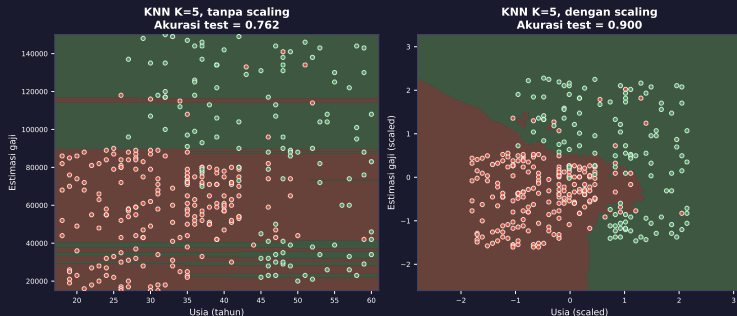
- Proporsinya bukan aturan baku; 60/20/20 dan 80/20 sama-sama umum.
- Kalau datanya sedikit, validation diganti cross-validation di data train.

- Tujuannya memprediksi respons nasabah sebelum telepon dilakukan.
- Kolom `duration` baru lengkap setelah panggilan berakhir.
- Memakai kolom itu membuat hasil evaluasi terlalu optimistis.

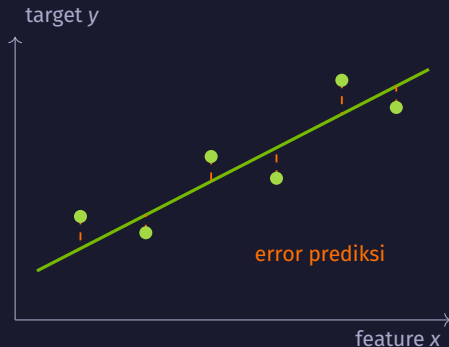
Ini salah satu bentuk leakage. Bentuk lain kita bahas di nbo2.



Scaling: menyamakan rentang antar feature



- Nilai gaji ada di puluhan ribu, umur di puluhan tahun. Tanpa scaling, selisih gaji mendominasi perhitungan jarak.
- Model berbasis jarak (KNN, SVM) dan model dengan penalti koefisien (logistic regression) butuh scaling; decision tree dan random forest tidak.



- Garis hijau adalah prediksi model, titik hijau muda adalah data sebenarnya.
- Garis putus-putus oranye adalah error prediksi tiap baris.
- Melatih model berarti mengubah parameternya sampai total error itu sekecil mungkin.

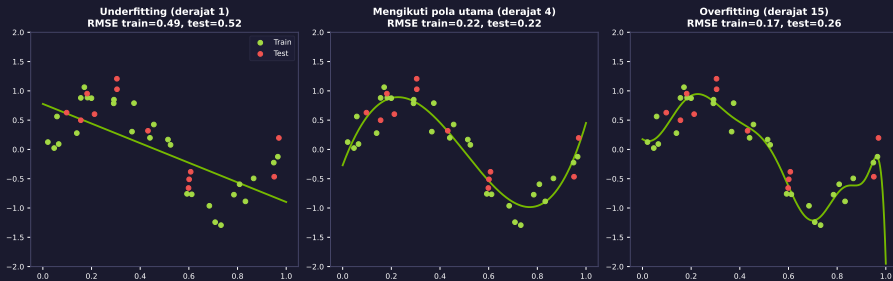
Loss: satu angka yang diminimalkan

Mean Squared Error

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

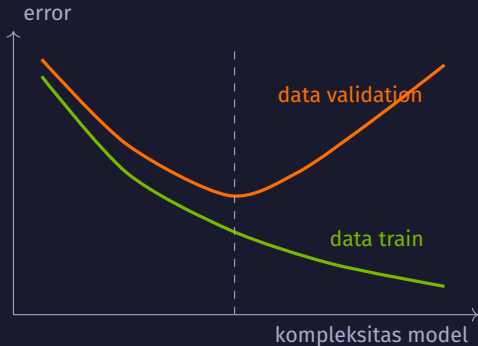
- y_i nilai sebenarnya, $\hat{y}_i$ prediksi model, n jumlah baris.
- Selisihnya dikuadratkan, jadi error besar menyumbang jauh lebih banyak daripada beberapa error kecil.
- Pelatihan mencari parameter dengan MSE terkecil di data train. Angka yang dilaporkan tetap diambil dari data yang tidak dipakai melatih.

Underfitting dan overfitting



- **Kiri:** garis lurus terlalu sederhana untuk pola yang melengkung. Error di data train dan data test dua-duanya besar.
- **Tengah:** masih mengikuti pola utama tanpa melewati semua titik.
- **Kanan:** mengikuti hampir setiap titik, termasuk variasi acaknya. Error train turun, error test naik.

Bias–variance trade-off



- Model yang terlalu sederhana dapat melewati pola.
- Model yang terlalu peka terhadap data train dapat berubah banyak ketika sampel pelatihannya berubah.
- Error di data train hampir selalu turun saat model dibuat lebih rumit. Yang perlu dilihat adalah error di data yang tidak dipakai melatih.

Cross-validation: menilai model lima kali

data train dibagi lima bagian sama besar



- Tiap putaran, satu bagian (oranye) dipakai menilai, empat sisanya untuk melatih.
- Skornya rata-rata lima putaran, jadi tidak bergantung pada satu potongan data yang kebetulan mudah.
- Data test tetap di luar semua ini.

Baseline: pembandingan paling sederhana

- **Regresi:** prediksi satu nilai konstan, yaitu rata-rata target di data train.
- **Klasifikasi:** prediksi kelas mayoritas untuk semua baris.
- Di `income_evaluation.csv`, 76% baris berlabel $\leq 50K$. Model yang memprediksi kelas itu untuk semua baris sudah dapat akurasi 76%.

Jebakan: akurasi 76% pada data seperti itu belum menunjukkan apa pun. Hitung baseline lebih dulu, lalu lihat selisih hasil modelmu terhadap baseline.

Metrik regresi: MAE, RMSE, R^2

Metrik	Arti
MAE	rata-rata selisih absolut prediksi dan nilai sebenarnya; satuannya sama dengan target
RMSE	akar dari rata-rata kuadrat error; lebih peka terhadap error besar daripada MAE karena perhitungannya memakai kuadrat error
R^2	membandingkan error kuadrat model dengan prediksi konstan berupa rata-rata target; 1 berarti prediksi tepat, 0 setara pembanding itu, negatif berarti lebih buruk

Pilih metrik sebelum melatih model, bukan setelah melihat hasilnya.

Confusion matrix: dua jenis kesalahan

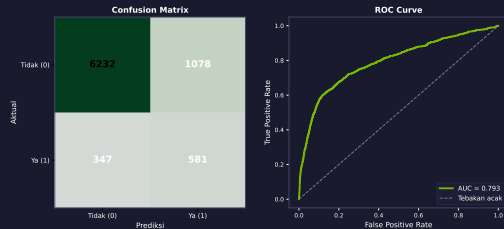
Positif berarti kondisi yang ingin dideteksi, bukan hasil yang baik.

		Prediksi: Tidak	Prediksi: Ya
Asli	Tidak	True Negative	False Positive alarm palsu
	Ya	False Negative kasus terlewat	True Positive

Akurasi menjumlahkan dua kotak hijau saja. Dua kotak sisanya punya biaya yang berbeda, dan itu yang menentukan metrik mana yang kita pakai.

Precision, recall, F1, dan ROC-AUC

- **Precision:** dari yang diprediksi positif, berapa bagian yang benar positif.
- **Recall:** dari yang benar-benar positif, berapa bagian yang tertangkap.
- **F1:** rata-rata harmonik precision dan recall, turun tajam kalau salah satu rendah.
- **ROC-AUC:** peluang model memberi skor lebih tinggi pada satu contoh positif dibanding satu contoh negatif.



Model logistic regression pada data banking tanpa duration.

Alur kerja yang dipakai di sepuluh notebook

1. Definisikan masalahnya dan metrik yang akan dilaporkan.
2. Siapkan data: periksa nilai kosong, outlier (pencilan), dan kolom yang tidak tersedia saat prediksi.
3. Split data train, validation, dan test.
4. Latih model di data train.
5. Bandingkan hasilnya dengan baseline.
6. Tuning hyperparameter dengan cross-validation di data train.
7. Uji sekali di data test, lalu laporkan angka itu.

Langkah 2 dan 3 memakan waktu paling banyak. Langkah 7 yang paling sering dilewati.

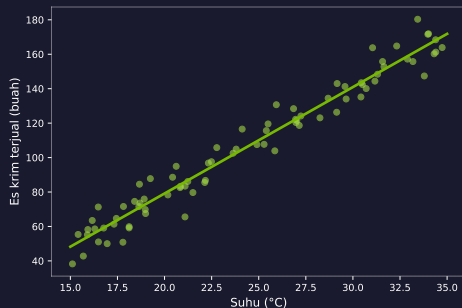
Act 3

Bagaimana memprediksi nilai?

Dari satu garis lurus sampai deret waktu dengan pola musiman

Suhu naik, es krim terjual lebih banyak

Intuisi



- Titik hijau adalah catatan harian: suhu hari itu dan jumlah es krim terjual.
- Banyak garis bisa ditarik di antara titik-titik itu. Yang dipilih hanya satu.

Intinya

Linear regression mencari garis yang total error prediksinya paling kecil.

Linear regression: satu garis, dua parameter

Bentuk modelnya

$$\hat{y} = wx + b$$

- w adalah kemiringan (slope): berapa banyak prediksi berubah tiap satu satuan kenaikan x .
- b adalah intercept: nilai prediksi saat $x = 0$.
- Dengan banyak feature, tiap feature punya satu w sendiri:
$$\hat{y} = w_1x_1 + w_2x_2 + \cdots + w_kx_k + b.$$
- Pelatihan memilih w dan b yang membuat MSE di data train terkecil.

Linear regression: kapan pakai dan jebakannya

Cocok kalau

- hubungan feature dan target mendekati lurus
- kamu perlu model yang koefisiennya bisa dibaca dan dijelaskan
- jumlah feature masih sedikit

Kurang cocok kalau

- polanya jelas melengkung atau berjenjang
- ada outlier (pencilan) yang menarik garisnya
- pengaruh satu feature bergantung pada feature lain

Jebakan: koefisien besar tidak berarti feature itu penyebabnya. Selain itu $x = 0$ sering berada di luar rentang data, jadi intercept belum tentu punya arti nyata.

Regularisasi: membatasi ukuran koefisien

Regularisasi (regularization) membatasi kompleksitas model agar tidak terlalu menyesuaikan diri dengan data train.

Yang diminimalkan

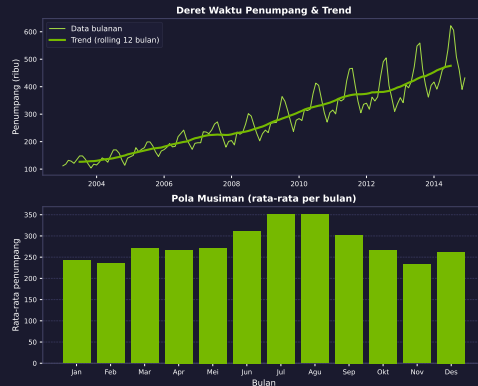
loss di data train + penalti atas ukuran koefisien

- **Ridge** mengecilkan semua koefisien, jarang sampai nol.
- **Lasso** bisa menolak koefisien yang tidak berguna, jadi sekaligus menyeleksi feature.
- Kekuatan penaltinya dipilih dengan cross-validation. Penalti yang terlalu kuat membuat model melewati pola penting.

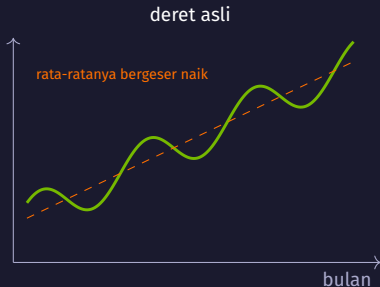
- **Trend:** arah jangka panjang, di sini naik terus.
- **Pola musiman:** naik-turun yang berulang tiap 12 bulan.
- **Noise:** sisa gerakan yang tidak dijelaskan keduanya.

Intinya

Urutan waktu adalah bagian dari datanya, jadi barisnya tidak boleh diacak seperti data tabular biasa.



Stasioner dan differencing

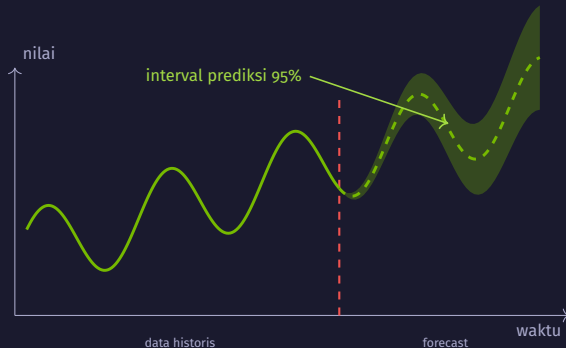


- Deret disebut **stasioner** kalau sifat statistiknya tetap terhadap pergeseran waktu. Grafik yang tampak datar belum membuktikannya.
- **Differencing** memakai selisih antar bulan sebagai deret baru, supaya rata-ratanya tidak lagi bergeser.

SARIMA dan interval prediksi

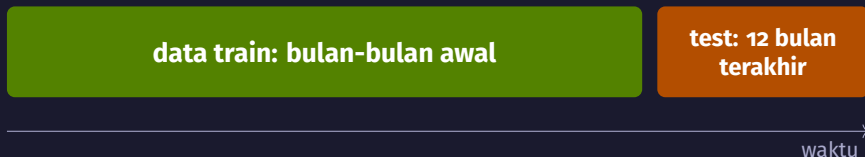
Seasonal AutoRegressive Integrated Moving Average

- **AR:** pakai nilai bulan-bulan sebelumnya.
- **I:** differencing seperti slide tadi.
- **MA:** pakai error prediksi sebelumnya.
- **S:** ulangi ketiganya pada jarak 12 bulan.



Pita hijau adalah interval prediksi: rentang untuk nilai bulan berikutnya. Cakupan 95% berlaku sejauh asumsi modelnya cocok dengan datanya.

Evaluasi time series: split kronologis



- Potongannya mengikuti waktu, bukan diacak. Model tidak boleh melihat bulan yang lebih baru daripada bulan yang diprediksi.
- **Baseline seasonal naive:** prediksi bulan ini sama dengan nilai bulan yang sama tahun lalu. SARIMA baru menarik kalau mengalahkan ini.
- **MAPE** adalah rata-rata error relatif dalam persen. Angkanya menyesatkan kalau nilai sebenarnya dekat nol.

Ringkasan Act 3

- Regresi memprediksi nilai berupa angka. Linear regression mencari w dan b yang membuat total error prediksi terkecil.
- Koefisien memberi arah dan besar pengaruh menurut model, bukan bukti sebab-akibat.
- Regularisasi menambah penalti atas ukuran koefisien; Lasso bisa menolak koefisien yang tidak berguna.
- Pada time series, urutan waktu ikut menentukan: split kronologis, differencing untuk menstabilkan rata-rata, dan seasonal naive sebagai baseline.

nb01 memakai Advertising, nb09 memakai SeaPlaneTravel.

Act 4

Bagaimana memprediksi kategori?

Empat model, empat cara menarik batas keputusan

Regresi dan klasifikasi

Regresi

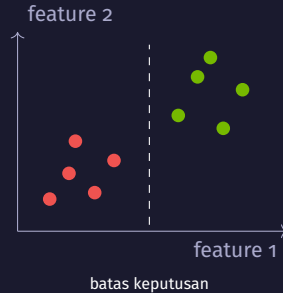


Keluarannya satu angka

lama waktu tempuh, penjualan

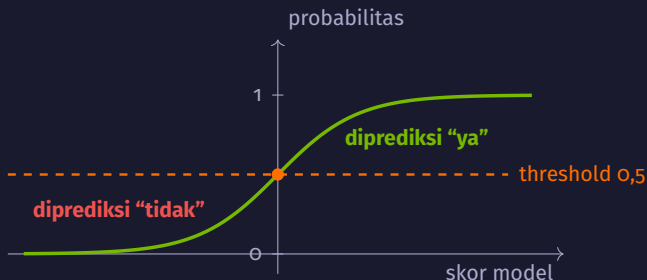
Alur kerjanya sama seperti Act 2. Yang berubah adalah bentuk keluaran model dan metrik yang dipakai menilainya.

Klasifikasi



Keluarannya label kelas

spam atau bukan, respons ya atau tidak



Intinya

Model menghitung satu skor dari semua feature, lalu fungsi sigmoid memampatkan skor itu menjadi angka antara 0 dan 1 yang dibaca sebagai probabilitas kelas positif.

Logistic regression: threshold dan data timpang

- Keluaran modelnya probabilitas. Label kelas muncul setelah probabilitas itu dibandingkan dengan sebuah threshold; `predict()` memakai 0,5.
- Menurunkan threshold menaikkan recall dan menurunkan precision. Menaikkannya melakukan yang sebaliknya. Jadi threshold adalah pilihan, bukan angka baku.
- Pilih thresholdnya dari data validation, mengikuti biaya kesalahan yang sudah disepakati.
- Kalau satu kelas jauh lebih banyak, `class_weight='balanced'` memperbesar bobot kelas minoritas saat melatih model.

Logistic regression: kapan pakai dan jebakannya

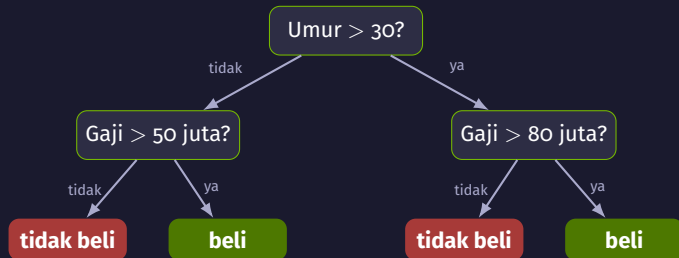
Cocok kalau

- kamu butuh probabilitas, bukan hanya label
- pengaruh tiap feature perlu bisa dijelaskan
- batas antar kelas mendekati garis lurus

Kurang cocok kalau

- batas antar kelas jelas melengkung
- banyak interaksi antar feature yang tidak kamu buat sendiri
- skala feature dibiarkan berbeda-beda tanpa scaling

Jebakan: akurasi 89% pada data yang 89% berlabel “tidak” sama saja dengan baseline kelas mayoritas. Lihat precision, recall, dan ROC-AUC sebelum menyimpulkan.



Satu baris data menuruni pohon lewat satu jalur saja, lalu memakai label di ujung jalur itu sebagai prediksi.

Decision tree: bagaimana pertanyaannya dipilih

- Di tiap simpul, model mencoba banyak pasangan feature dan ambang, lalu memilih pasangan yang paling memisahkan kelas.
- Ukuran pemisahannya adalah **gini**: makin kecil, makin murni isi tiap cabang.
- Proses itu diulang pada tiap cabang sampai simpulnya murni atau batas yang kamu pasang tercapai.
- Batasnya yang penting: `max_depth` (kedalaman) dan `min_samples_leaf` (isi minimum tiap ujung).

Karena tiap pertanyaan hanya membandingkan satu feature dengan satu angka, batas keputusannya selalu sejajar sumbu feature.

Decision tree: kapan pakai dan jebakannya

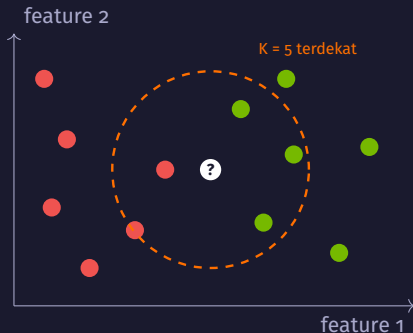
Cocok kalau

- aturan hasilnya perlu dibaca orang lain
- feature bercampur angka dan kategori
- scaling tidak dilakukan; pohon tidak membutuhkannya

Kurang cocok kalau

- batas antar kelas miring atau melengkung halus
- datanya sedikit dan pohonnya dibiarkan tumbuh bebas
- kamu butuh prediksi yang stabil terhadap perubahan kecil data

Jebakan: tanpa batas kedalaman, pohon dapat menghafal data train sampai errornya nol dan hasilnya jatuh di data baru. Pilih kedalamannya dengan cross-validation.



- Titik putih adalah data baru yang belum punya label.
- Lima data train terdekat: tiga hijau, dua merah. Prediksinya hijau.

Intinya

Tetangga di sini berarti data train yang paling dekat menurut jarak pada feature, bukan tetangga dalam arti sehari-hari.

KNN: jarak, K, dan suara mayoritas

Jarak Euclidean antara dua baris data

$$d(a, b) = \sqrt{\sum_{j=1}^m (a_j - b_j)^2}$$

- a dan b dua baris data, m jumlah feature, dan K jumlah tetangga yang dilihat.
- Hitung jarak data baru ke data train, ambil K yang terdekat, lalu pakai kelas terbanyak di antara mereka.
- K kecil membuat prediksi mengikuti tiap variasi acak di sekitarnya. K besar membuat batasnya terlalu rata dan pola lokal hilang.
- Tidak ada parameter yang dilatih. Semua pekerjaan terjadi saat prediksi.

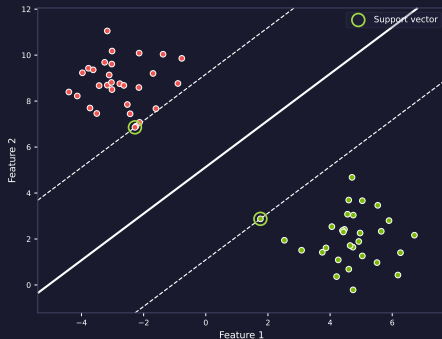
KNN: kapan layak dicoba

- Mulai dari data kecil dengan sedikit feature yang relevan.
- Prediksi dapat melambat saat data train membesar, karena pencarian tetangga dilakukan setiap kali memprediksi.
- Jika skala feature berbeda, lakukan scaling agar perhitungan jarak tidak didominasi satu feature.

Jebakan: gaji ditulis dalam juta rupiah dan umur dalam tahun. Tanpa scaling, selisih gaji yang rentang nilainya jauh lebih besar hampir menentukan seluruh jaraknya.

SVM: cari jalan paling lebar di antara dua kelas

Intuisi



- Di antara semua garis pemisah, SVM memilih yang jarak ke kelas terdekat di kedua sisinya paling lebar.
- Titik yang menempel di tepi jalan itu disebut support vector.

Intinya

Hanya titik di dekat batas yang menentukan posisi garisnya. Titik yang jauh boleh digeser tanpa mengubah hasilnya.

SVM: margin, C, dan kernel

- **Margin** adalah lebar jalan antara batas keputusan dan support vector terdekat. Pelatihan memaksimalkan lebar itu.
- **C** mengatur seberapa banyak titik boleh masuk ke dalam margin. C kecil memberi margin lebar dengan beberapa pelanggaran; C besar memaksa margin sempit yang rapat mengikuti data train.
- **Kernel RBF** mengangkat data ke ruang berdimensi lebih tinggi, tempat kedua kelas dapat dipisahkan bidang datar. Kembali ke dua feature semula, batasnya terlihat melengkung.
- γ pada RBF mengatur seberapa lokal pengaruh satu titik.

SVM: kapan pakai dan jebakannya

Cocok kalau

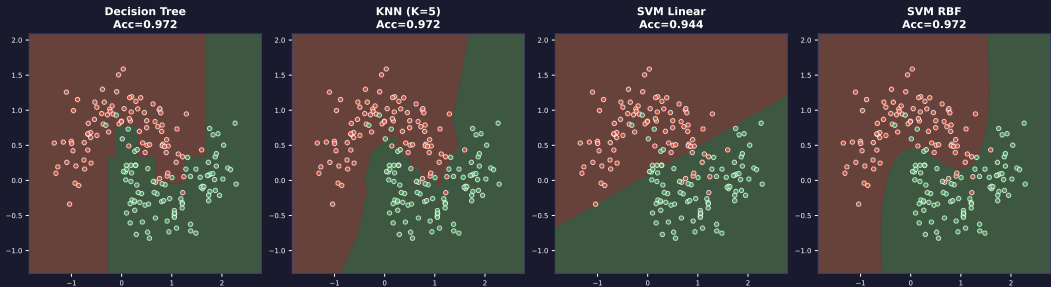
- jumlah feature banyak dibanding jumlah baris
- batas antar kelas cukup tegas
- kamu bersedia mencari pasangan C dan γ lewat cross-validation

Kurang cocok kalau

- data trainnya ratusan ribu baris
- probabilitas kelas dibutuhkan langsung dari model
- hasilnya harus dijelaskan sebagai aturan per feature

Jebakan: kernel RBF melambat cepat saat data membesar, dan SVM mengukur jarak. Jika skala feature berbeda, lakukan scaling lebih dulu.

Empat model, satu dataset



- Data yang sama, empat batas keputusan yang berbeda: bentuk batasnya mengikuti cara model itu menarik keputusan.
- Karena itu perbandingan model selalu dilakukan pada satu split dan satu metrik yang sama.

Ringkasan Act 4: kapan pakai model yang mana

Model	Bentuk batas keputusan	Scaling	Prediksi	Dibaca orang
Logistic regression	garis lurus	perlu	cepat	mudah
Decision tree	sejajar sumbu feature	tidak perlu	cepat	mudah
KNN	mengikuti sebaran data	perlu	melambat	sulit
SVM linear	garis lurus	perlu	sedang	sedang
SVM RBF	melengkung	perlu	lambat	sulit

Kolom scaling berlaku kalau skala feature berbeda-beda. Tabel ini membantu memilih kandidat awal; keputusannya tetap dari hasil di data validation.